

Bloque 07_redes_y_pivoting

Índice de módulos

- Enumeracion de DNS
- Fingerprinting antes que el nombre del puerto
- Enumeracion de FTP
- Segmentacion, rutas y firewalls
- Trafico de red, poisoning y relay
- Enumeracion de NFS
- Captura y analisis de paquetes
- Diagnóstico de pivoting
- Pivoting: tuneles a la red interna
- Metodologia de reconocimiento
- Enumeracion de SMB
- Enumeracion de SMTP
- Enumeracion de SNMP
- TCP frente a UDP

Enumeracion de DNS

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Enumerar DNS es sacarle al servidor de nombres toda la información de dominios y subdominios que gestiona: registros A/AAAA (IPs), MX (correo), TXT (a veces con pistas), y sobre todo intentar una transferencia de zona (AXFR), que si esta mal configurada te da TODOS los registros de golpe, como si te regalaran el mapa completo.

AXFR existe para que servidores DNS secundarios se sincronicen con el primario, pero si el servidor no restringe quien puede pedirla, cualquiera puede solicitarla y recibir el listado completo de subdominios/hosts internos. Es una mala configuración clásica, no una vulnerabilidad del protocolo en si.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando el recon muestra el puerto 53 abierto, o cuando conoces un dominio del objetivo (visto en un certificado TLS, un email, o la propia room) y quieres descubrir subdominios/hosts internos asociados.

Cómo identificarla

- Puerto 53 (TCP/UDP) abierto en el objetivo.
- Un hostname en un certificado TLS o un email que revela el dominio interno a enumerar.
- dig axfr no devuelve REFUSED sino una lista de registros.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Intenta una transferencia de zona (AXFR) contra el servidor DNS del dominio conocido.

Evidencia esperada: Si funciona, una lista completa de registros (subdominios, IPs internas, hostnames) en la respuesta. Si falla, el servidor la tiene bien configurada; pasa a fuerza bruta de subdominios.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
dig axfr @$IP dominio.local
```

Objetivo y contexto: Intenta la transferencia de zona directamente. Es la prueba mas rentable: si el servidor la permite, obtienes todos los registros sin fuerza bruta.

Resultado esperado: Si funciona, una lista completa de registros A/CNAME/MX con nombres de host internos. 'Transfer failed' o REFUSED significa que esta bien configurado.

Anatomía de los payloads

La primera prueba es `dig axfr @$IP dominio.local`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Si funciona, una lista completa de registros (subdominios, IPs internas, hostnames) en la respuesta. Si falla, el servidor la tiene bien configurada; pasa a fuerza bruta de subdominios.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Prueba AXFR directamente contra el servidor DNS del dominio conocido; es rapido y, si funciona, te da todo de golpe.
- Si falla, haz fuerza bruta de subdominios con una wordlist y anade los que resuelvan a /etc/hosts.
- Cruza los hostnames encontrados con vhosts web (1fi y el resto de bugs web pueden variar por vhost).
- Si el objetivo es un DC de Active Directory, el DNS suele revelar el nombre del dominio y del propio DC: mira ad-modelo.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor rechaza la transferencia de zona (respuesta 'Transfer failed' o REFUSED) y no conoces mas nombres para probar por fuerza bruta.
- No hay ningun dominio conocido ni servidor DNS accesible para el objetivo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El DNS revela subdominios, hostnames internos y a veces todo el mapa de la zona si el servidor permite una transferencia. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (dns - enum)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Fingerprinting antes que el nombre del puerto

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

IANA y Nmap pueden asociar un puerto a un nombre habitual, pero cualquier proceso puede escuchar ahí.

Fingerprinting es identificar lo que realmente responde mediante handshake, cabeceras, banner, TLS y detección de versión.

Asumir que 5050 es 'mmcc' puede hacer que ignores una web Flask o Werkzeug. El servicio real determina tus siguientes pruebas: HTTP se enumera con rutas, formularios y cabeceras; SSH con usuarios y credenciales; no con el nombre histórico del puerto.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Siempre que encuentres un puerto no estándar, un banner incompleto o una asociación de Nmap que no encaje con la respuesta observada.

Cómo identificarla

- Un puerto alto o raro responde con cabeceras HTTP.
- El nombre de servicio de Nmap no coincide con el banner o con el comportamiento.
- -sV identifica un producto/framework distinto de la asociación por puerto.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Nmap -sV, curl o una conexión de protocolo devuelve una respuesta coherente con el servicio real.

Evidencia esperada: Tienes protocolo, producto y versión para decidir la enumeración y buscar vulnerabilidades aplicables.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
nmap -sC -sV -Pn -p <PUERTO> $IP
```

Objetivo y contexto: Confirma que habla realmente el puerto y recoge scripts/banners utiles.

Resultado esperado: Servicio, producto, version y pistas como titulos HTTP o certificados.

Capa 2: prueba documentada

```
curl -i http://$IP:<PUERTO>/
```

Objetivo y contexto: Una comprobacion HTTP directa separa una aplicacion web real de una etiqueta de puerto equivocada.

Resultado esperado: Status, cabeceras y posiblemente framework o rutas a enumerar.

Anatomía de los payloads

La primera prueba es `nmap -sC -sV -Pn -p <PUERTO> $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tienes protocolo, producto y version para decidir la enumeracion y buscar vulnerabilidades aplicables.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Registra el puerto abierto sin asumir su funcion.
- Ejecuta deteccion de version y observa la respuesta de protocolo.
- Enumera segun el servicio confirmado y no segun el alias del puerto.
- Guarda producto y version exactos como evidencia para el siguiente paso.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No sustituyas una identificacion de version fiable por el nombre del puerto.
- No concluyas que un servicio esta ausente solo porque no usa su puerto por defecto.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El nombre asignado a un puerto es una pista, no una identidad: confirma protocolo, producto y version antes de elegir la enumeracion. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (fingerprinting-servicios)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion de FTP

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

FTP (puerto 21) es un protocolo de transferencia de ficheros anterior a la web. Muchas instalaciones dejan habilitado el login 'anonymous' (usuario anonymous, cualquier password) por comodidad o por defecto. Si esta activo, entras sin credenciales y puedes listar, descargar, y a veces subir ficheros segun los permisos del share.

El login anonimo es una funcionalidad legitima de FTP para descargas publicas, pero cuando se deja activo sobre un share que contiene ficheros sensibles (backups, configs, notas), se convierte en una fuga de informacion gratuita. Es la misma logica que la sesion nula de smb-enum aplicada a FTP.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Siempre que el recon muestre el puerto 21 abierto. Es de las comprobaciones mas baratas: un intento de login anonimo tarda segundos.

Cómo identificarla

- Puerto 21 abierto en el escaneo de puertos.
- El banner de nmap/ftp menciona 'Anonymous FTP login allowed'.
- Login con usuario anonymous aceptado.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Conecta con usuario anonymous y password vacia o cualquiera; si el login es aceptado, lista el contenido.

Evidencia esperada: Un login exitoso (230 Login successful) seguido de un listado de ficheros/directorios confirma acceso anonimo.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

ftp \$IP

Objetivo y contexto: Conecta al servicio FTP para probar el login anonimo de forma interactiva. Cuando pida usuario, prueba anonymous; password, cualquier cosa o vacia.

Resultado esperado: '230 Login successful' confirma acceso anonimo. Un ls/dir tras entrar te muestra el contenido del share; get descarga ficheros.

Anatomía de los payloads

La primera prueba es ftp \$IP. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un login exitoso (230 Login successful) seguido de un listado de ficheros/directorios confirma acceso anonimo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Prueba login anonimo primero; es gratis y muy comun que funcione en labs.
- Si entra, lista y descarga TODO lo accesible; busca configs, backups, claves SSH o notas con credenciales.
- Comprueba tambien permisos de escritura (STOR): un FTP anonimo con escritura permite subir un webshell si el mismo directorio se sirve por HTTP.
- Cualquier credencial encontrada -> pruebala en otros servicios (cracking si esta hasheada, reuse si esta en claro).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor rechaza el usuario anonymous y no tienes ninguna otra credencial que probar.
- El share anonimo esta vacio o solo contiene ficheros publicos sin valor.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

FTP suele permitir login anónimo; si lo hace, listas y descargas ficheros sin credenciales, y a veces hasta escribes. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(ftp-enum\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Segmentacion, rutas y firewalls

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

La segmentacion separa redes y limita sus flujos mediante rutas, ACL y firewalls. Alcanzable describe que existe un camino; permitido describe que los controles dejan pasar protocolo, puerto y direccion concretos.

Un host comprometido puede tener rutas que Kali no posee. Tambien puede aceptar conexiones salientes pero bloquear entrantes, motivo por el que un tunel reverse funciona donde uno directo falla.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando aparecen varias interfaces, subredes internas, puertos filtered, respuestas asimetricas o servicios accesibles solo desde otro host.

Cómo identificarla

- ip route
- Dos interfaces
- filtered
- Timeout
- Egress permitido
- Red interna

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara rutas e interfaces desde ambos extremos y prueba un unico host/puerto conocido antes de lanzar enumeracion amplia.

Evidencia esperada: Un mapa de origen, destino, ruta, direccion permitida y punto de filtrado que justifica si necesitas pivot o port forwarding.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`ip route`

Objetivo y contexto: Muestra que redes conoce el host actual y por que interfaz o gateway intentara alcanzarlas.

Resultado esperado: Prefijos, gateways e interfaces que revelan redes internas y posibles pivots.

Anatomía de los payloads

La primera prueba es `ip route`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un mapa de origen, destino, ruta, direccion permitida y punto de filtrado que justifica si necesitas pivot o port forwarding.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Dibuja redes por host.
- Compara las rutas.
- Prueba un puerto conocido.
- Identifica direccion y control.
- Elige tunel o forwarding minimo.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Closed indica que el host respondió pero no hay servicio; filtered suele indicar ausencia de respuesta o filtrado y exige otra hipótesis.
- Un tunel no concede permisos de aplicacion: solo cambia la conectividad.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Distinguir que redes conoce cada host, que camino sigue el tráfico y que control permite o bloquea cada dirección. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (network-segmentation-firewalls)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Trafico de red, poisoning y relay

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Observar trafico describe protocolos y flujos; poisoning responde a resoluciones inseguras para provocar autenticacion; relay reenvia esa autenticacion a otro servicio. Son niveles de intervencion y riesgo diferentes.

LLMNR/NBT-NS y NTLM pueden exponer hashes NetNTLM o permitir relay cuando la firma SMB no es obligatoria. La segmentacion y los controles de firma determinan si el ataque atraviesa la red.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En un segmento de laboratorio donde aparecen broadcasts de resolucion, autenticacion NTLM y objetivos con SMB signing desactivado.

Cómo identificarla

- LLMNR
- NBT-NS
- NetNTLMv2
- SMB signing disabled
- Broadcast
- Segmentacion

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Separa evidencia pasiva, hash capturado y autenticacion reenviada; no presentes una captura como si ya implicara ejecucion.

Evidencia esperada: Sabes si existe solo exposicion de credenciales, posibilidad de crackeo offline o una ruta de relay con impacto lateral.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
sudo responder -I tun0 -wv
```

Objetivo y contexto: Escucha y responde en la interfaz autorizada para capturar autenticación de laboratorio.

Resultado esperado: Hash NetNTLMv2 con usuario y host de origen.

Capa 2: prueba documentada

```
impacket-ntlmrelayx -tf targets.txt -smb2support
```

Objetivo y contexto: Prueba relay solo contra objetivos previamente validados sin firma SMB obligatoria.

Resultado esperado: Autenticación aceptada o rechazo que confirma el control defensivo.

Anatomía de los payloads

La primera prueba es `sudo responder -I tun0 -wv`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Sabes si existe solo exposición de credenciales, posibilidad de crackeo offline o una ruta de relay con impacto lateral.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica interfaz y alcance.
- Observa antes de responder.
- Captura una autenticación de laboratorio.
- Comprueba signing en objetivos.
- Solo entonces evalúa relay y documenta impacto.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No estas en el mismo segmento ni tienes un pivot con visibilidad del broadcast.
- SMB signing es obligatorio en los objetivos de relay.
- El alcance no autoriza poisoning o manipulacion de trafico.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Distinguir observacion pasiva, captura de autenticacion y relay activo dentro del mismo segmento. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (network-traffic-mitm)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion de NFS

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

NFS (puerto 2049) comparte directorios del servidor para que otras maquinas los monten como si fueran locales. Antes de montar nada, puedes preguntarle al servidor QUE exporta y con que restricciones (showmount). Si un export no restringe por host o usuario, lo montas y navegas sus ficheros como si estuvieras en el servidor.

NFS confía en el UID/GID que le manda el CLIENTE para decidir permisos, en vez de autenticar de verdad; si tu Kali usa el mismo UID que el dueño de los ficheros en el servidor (o el export no restringe), lees y a veces escribes sin credenciales. Es la base también de nfs-no-root-squash, la variante que da privesc directo.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando el recon muestra el puerto 2049 (o 111 portmapper) abierto. Compruébalo siempre antes de intentar montar nada a ciegas.

Cómo identificarla

- Puerto 2049 (NFS) o 111 (portmapper) abierto en el escaneo.
- showmount -e devuelve al menos un export sin restricción de host.
- Al montar, los ficheros muestran UID/GID numéricos en vez de nombres (no coinciden con tus usuarios locales).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pide la lista de exports con showmount -e contra la IP objetivo.

Evidencia esperada: Una lista de rutas exportadas (y sus restricciones de host, si las hay). Un export accesible para ti (* o tu rango) confirma que puedes montarlo.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
showmount -e $IP
```

Objetivo y contexto: Pregunta al servidor NFS que exporta y a quien, sin montar nada todavía. Es el primer paso obligado antes de montar a ciegas.

Resultado esperado: Rutas exportadas y su restricción de host (o * si es abierto a todos). Cada ruta accesible es un candidato a montar y explorar.

Capa 2: prueba documentada

```
sudo mount -t nfs $IP:/export /mnt/nfs
```

Objetivo y contexto: Monta el export localmente para navegarlo como un directorio normal. Requiere sudo porque montar sistemas de ficheros es una operación privilegiada en tu propia máquina.

Resultado esperado: El contenido del export disponible en /mnt/nfs. Explora en busca de ficheros de config, claves SSH, o binarios que puedas modificar.

Anatomía de los payloads

La primera prueba es `showmount -e $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una lista de rutas exportadas (y sus restricciones de host, si las hay). Un export accesible para ti (* o tu rango) confirma que puedes montarlo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lista los exports disponibles con `showmount -e` antes de montar nada.
- Monta el export en un punto local y navega su contenido como cualquier directorio.
- Fijate en los UID/GID de los ficheros: si no coinciden con tu usuario, quizá necesites ajustar tu propio UID para leer/escribir como el dueño.
- Si el export tiene `no_root_squash`, no hace falta ajustar UID de lectura: puedes escalar directamente, ve a `nfs-no-root-squash`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- showmount -e no devuelve ningún export, o todos están restringidos a hosts que no eres tú.
- No tienes forma de montar NFS desde tu máquina de ataque.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

NFS lista que directorios exporta y a quien; montarlos revela ficheros del servidor directamente en tu Kali. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (nfs -enum)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Captura y analisis de paquetes

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Una captura PCAP conserva paquetes observados en una interfaz. Los filtros de captura reducen que se guarda; los filtros de visualización seleccionan después que analizar. Seguir un stream reconstruye la conversación de una sesión TCP.

Mirar paquetes aislados produce conclusiones pobres. Relacionar DNS, establecimiento TCP, protocolo de aplicación, autenticación y errores permite identificar hosts, servicios y datos expuestos con evidencia temporal.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando tienes permiso y visibilidad de una interfaz, una PCAP de una room o necesitas entender por qué dos sistemas no se comunican.

Cómo identificarla

- DNS
- SYN/SYN-ACK
- Retransmission
- HTTP
- SMB2
- Kerberos
- Follow stream

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Filtra por protocolo o host, sigue el stream relevante y relaciona cada conclusión con paquetes y marcas de tiempo.

Evidencia esperada: Una secuencia explicable de quien inició la conexión, que protocolo usó, que respondió el destino y donde falló.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
tshark -r capture.pcap -Y 'dns or http or kerberos or smb2'
```

Objetivo y contexto: Aplica un filtro de visualización reproducible sobre una captura ya obtenida sin modificar el tráfico.

Resultado esperado: Paquetes relevantes con tiempo, origen, destino, protocolo y resumen.

Anatomía de los payloads

La primera prueba es `tshark -r capture.pcap -Y 'dns or http or kerberos or smb2'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una secuencia explicable de quien inicio la conexión, que protocolo uso, que respondió el destino y donde fallo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma interfaz y tiempo.
- Captura con el menor filtro necesario.
- Aplica filtros de visualización.
- Sigue la conversación.
- Registra paquetes que sustentan la conclusión.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No ver tráfico no demuestra que el servicio no exista: puede faltar visibilidad, interfaz o ventana temporal.
- TLS protege el contenido aunque metadatos como IP, puerto, SNI y tiempos sigan siendo visibles.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.

- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Separar captura, filtrado e interpretación para reconstruir conversaciones de red sin intervenir en ellas. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(packet-analysis\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Diagnóstico de pivoting

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Pivoting depende de que la ruta, el proceso de tunel y el proxy conozcan correctamente cada red.

Un tunel activo no garantiza que el trafico use la interfaz o el puerto correctos.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ves la maquina pivote pero un servicio interno da timeout o connection refused.

Cómo identificarla

- timeout
- proxy error
- route missing
- listener activo

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Comprueba interfaz interna, ruta, escucha del proxy y conexion desde el punto correcto.

Evidencia esperada: Sabes si falla el tunel, el proxy, la ruta o el servicio objetivo.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
ip addr; ip route
```

Objetivo y contexto: Revela interfaces y redes que realmente ve el pivote.

Resultado esperado: Subred interna, gateway y rutas disponibles.

Capa 2: prueba documentada

```
ss -ltnp
```

Objetivo y contexto: Confirma que el servicio de tunel o proxy escucha donde esperas.

Resultado esperado: Puerto, interfaz y proceso propietario.

Capa 3: prueba documentada

```
proxychains -q nmap -Pn -p <PUERTO> <IP_INTERNA>
```

Objetivo y contexto: Valida el camino completo hacia un servicio interno.

Resultado esperado: open, filtered o timeout a través del proxy.

Anatomía de los payloads

La primera prueba es `ip addr; ip route`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Sabes si falla el tunel, el proxy, la ruta o el servicio objetivo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Inspecciona interfaces.
- Comprueba rutas y listener.
- Prueba desde el pivote y desde proxychains.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No aplica si el objetivo interno es directamente alcanzable desde tu máquina.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Comprobar rutas, listeners y alcance cuando el tunel existe pero no llega al servicio interno. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (pivot-troubleshooting)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Pivoting: tuneles a la red interna

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Pivotar es usar una maquina ya comprometida como puente para llegar a hosts internos que tu atacante no alcanza directamente. Se monta un tunel (SOCKS) a traves del pivot y se enruta el trafico de tus herramientas por el, con lo que puedes escanear y atacar la red interna como si estuvieras dentro.

Las redes se segmentan: el pivot tiene una segunda interfaz o rutas a rangos que tu no ves. Un tunel reverse (chisel, ligolo-ng) hace que el pivot conecte hacia ti, esquivando firewalls que bloquean entrada. Por un SOCKS solo pasa TCP, de ahí que en nmap uses -sT y no SYN scan.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ya tienes acceso a una maquina y descubres una red interna nueva (ip a, ip route, arp -a muestran rangos que tu Kali no tiene). Cada host interno alcanzado reinicia el ciclo recon/enum/acceso.

Cómo identificarla

- El pivot tiene una IP en un rango distinto al que tu atacas (ej. 10.10.20.x).
- ip route o arp -a revelan hosts internos que no salían en tu escaneo inicial.
- Servicios internos que solo responden a traves del proxy.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Levanta el tunel y prueba un curl/nc contra un puerto conocido de la red interna antes de escanear.

Evidencia esperada: El puerto SOCKS local escuchando (ss -tlnp) y respuesta del host interno a traves del proxy.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
ssh -D 1080 -N user@$IP
```

Objetivo y contexto: Si tienes SSH al pivot, -D monta un proxy SOCKS local sin instalar nada. Es la vía más rápida y limpia con credenciales SSH válidas.

Resultado esperado: No imprime nada (-N): el éxito es el puerto 1080 escuchando en tu Kali (compruébalo con `ss -tlnp`). A partir de ahí, todo va con `proxychains`.

Capa 2: prueba documentada

```
proxychains nmap -sT -Pn -n -p80,445 10.10.20.5
```

Objetivo y contexto: `proxychains` rutea `nmap` por el SOCKS hacia la red interna. -sT (connect) es obligatorio: por un SOCKS no pasa el SYN scan y darías todo por cerrado.

Resultado esperado: Puertos abiertos del host interno. Ve a pocos puertos (es lento por el túnel). Servicios abiertos = una mini-room nueva dentro de la red.

Anatomía de los payloads

La primera prueba es `ssh -D 1080 -N user@$IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El puerto SOCKS local escuchando (`ss -tlnp`) y respuesta del host interno a través del proxy.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma la red interna desde el pivot (`ip a`, `ip route`, `arp -a`).
- Monta el túnel: `ssh -D` si tienes SSH, o `chisel`/`ligolo` en reverse si solo ejecutas binarios.
- Enruta tus herramientas por el SOCKS con `proxychains` (`nmap` con -sT).
- Valida con un objetivo pequeño y repite el ciclo de ataque contra los hosts internos.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `ip route/arp -a` no muestran ninguna red distinta a la que ya atacas: no hay nada que pivotar.
- No tienes forma de ejecutar binarios en el pivot (solo lectura) y tampoco tienes SSH con -D disponible.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

La maquina comprometida ve una red que tu Kali no alcanza; la usas de puente montando un tunel. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (pivoting-tunnel)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Metodología de reconocimiento

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

El reconocimiento es cartografiar la superficie de ataque: que puertos están abiertos, que servicios corren y en qué versión. Se hace en capas de lo amplio a lo profundo para no perder tiempo ni hacer ruido: primero todos los puertos sin scripts, después versión y scripts solo en los abiertos, y por último enumeración específica de cada servicio.

Escanear versiones de los 65535 puertos es lentísimo e inútil. Separar 'descubrir puertos' de 'analizar servicios' hace el recon rápido y ordenado. Y cada servicio se enumera distinto, así que primero necesitas saber cuáles hay antes de elegir la técnica.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Es SIEMPRE la primera fase de una room. Cada servicio que descubras deriva a su enumeración propia (445 -> smb-enum, 80 -> web, 88/389 -> ad-modelo).

Cómo identificarla

- Un puerto abierto con una versión concreta: tu primera pista para buscar CVEs.
- Hostnames en certificados TLS o en scripts de nmap: anádelos a /etc/hosts.
- Servicios en puertos no estándar (labs que los mueven a propósito).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara el número de puertos abiertos entre el barrido rápido y el escaneo con -sV: deben coincidir.

Evidencia esperada: Una lista de puertos open, y en el segundo escaneo, versión y banner de cada servicio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
nmap -p- -Pn -n --min-rate 5000 $IP -oN nmap/all_ports.txt
```

Objetivo y contexto: Barrido de amplitud: los 65535 puertos sin scripts es rapido. -Pn evita que nmap descarte el host por no responder al ping (comun en labs).

Resultado esperado: Solo la lista de puertos open. Copia esos numeros para el segundo escaneo; ignora versiones aqui.

Capa 2: prueba documentada

```
nmap -sC -sV -Pn -p <PUERTOS> $IP -oN nmap/services.txt
```

Objetivo y contexto: Profundidad solo donde hace falta: version exacta (-sV, base de los CVEs) y scripts por defecto (-sC) que sacan datos gratis.

Resultado esperado: Versiones, banners, hostnames en certificados y hallazgos de scripts (smb anonimo, titulos web). Cada version -> searchsploit.

Anatomía de los payloads

La primera prueba es `nmap -p- -Pn -n --min-rate 5000 $IP -oN nmap/all_ports.txt`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una lista de puertos open, y en el segundo escaneo, version y banner de cada servicio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Barrido de los 65535 puertos sin scripts, rapido, para saber donde mirar.
- Segundo escaneo con -sC -sV SOLO sobre los puertos abiertos, para versiones y datos gratis.
- Asocia cada puerto a su enumeracion (web, SMB, DNS, dominio) y profundiza.
- Anota versiones y hostnames; alimentan la busqueda de CVEs y el resto del ataque.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ya tienes la lista completa de puertos/servicios de una fase anterior (no repitas el barrido).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

No puedes atacar lo que no sabes que existe: primero puertos (amplio y rapido), luego versiones, luego enum por servicio. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (recon-metodologia)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion de SMB

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

SMB (445, y 139) es el protocolo de compartición de ficheros de Windows. Enumerarlo consiste en listar sus shares (carpetas compartidas), ver a cuáles puedes acceder sin o con credenciales, y sacar información del sistema y del dominio. Es una de las superficies más rentables en rooms Windows/AD.

Muchas configuraciones permiten sesión nula o de invitado, dejando shares legibles con backups, configs o credenciales. Y aunque no haya acceso anónimo, SMB filtra el nombre del equipo, el dominio y a veces la lista de usuarios, que alimentan el resto del ataque (spraying, kerberos).

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Siempre que el recon muestre 445 abierto. Es de las primeras cosas a enumerar; en un DC además se combina con LDAP y Kerberos (ad-modelo).

Cómo identificarla

- nmap marca 445/tcp open microsoft-ds o el script smb saca el dominio.
- Un share con permiso READ que no sea IPC\$ (a menudo backups, transfer, dev).
- Sesión nula/guest aceptada: puedes listar sin credenciales.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

`nxc smb $IP -u " " -p " --shares` para probar sesión nula.

Evidencia esperada: Columna READ/WRITE por share; un READ fuera de IPC\$ es acceso real. El header confirma nombre de equipo y dominio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
nxc smb $IP -u '' -p '' --shares
```

Objetivo y contexto: Comprueba sesión nula y lista los shares de golpe. Es la prueba más rentable en SMB: un share legible te ahorra toda la fase de explotación.

Resultado esperado: Columna READ/WRITE por share. Un READ fuera de IPC\$ es saqueo directo. El encabezado también confirma nombre de equipo y dominio.

Capa 2: prueba documentada

```
nxc smb $IP -u 'guest' -p '' --rid-brute
```

Objetivo y contexto: Si hay acceso guest/nulo, itera los RID para sacar la lista de usuarios del dominio sin credenciales. Necesitas usuarios válidos antes de rociar passwords.

Resultado esperado: Líneas SidTypeUser = usuarios reales. Extrae los nombres (sin las cuentas de máquina terminadas en \$) a users.txt para spraying y roasting.

Anatomía de los payloads

La primera prueba es `nxc smb $IP -u '' -p '' --shares`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Columna READ/WRITE por share; un READ fuera de IPC\$ es acceso real. El header confirma nombre de equipo y dominio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba acceso anónimo listando shares con usuario vacío o guest.
- Entra a cada share legible y saquea configs, backups y ficheros de usuario.
- Si tienes credenciales, repite: suele abrir más shares y, si eres admin local, ejecución.
- Extrae usuarios (rid-brute) para alimentar spraying y roasting en AD.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- SMB firmado y sin sesión nula/guest, y no tienes ninguna credencial que probar.

- El servicio en 445 no es realmente SMB (honeypot o servicio remapeado).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El puerto 445 suele regalar shares legibles, nombres de usuario y el dominio; comprueba siempre el acceso anonimo primero. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(smb-enum\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion de SMTP

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

SMTP (puerto 25, o 587/465) es el protocolo de envío de correo. Algunos servidores mal configurados responden a los comandos VRFY (verifica si un usuario existe) o EXPN (expande una lista de distribución), lo que te deja confirmar usuarios validos del dominio sin autenticarte. El banner tambien suele filtrar el software y version del MTA.

VRFY/EXPN son comandos legitimos del protocolo SMTP pensados para diagnostico, pero dejarlos activos sin restriccion permite enumerar cuentas de correo validas (que a menudo son tambien cuentas del sistema o del dominio). Es la misma logica de fuga de informacion que RID brute en smb-enum.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando el recon muestra el puerto 25 (o 587/465) abierto. Sirve tanto para sacar usuarios como para entender que servidor de correo corre.

Cómo identificarla

- Puerto 25/587/465 abierto en el escaneo.
- El banner de conexion revela el software MTA y su version.
- VRFY responde con codigos distintos segun si el usuario existe o no.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Conecta por SMTP y prueba VRFY con un nombre de usuario probable (root, admin, un nombre visto en otra fuente).

Evidencia esperada: Un codigo 250/252 (usuario existe) frente a 550 (no existe) te deja distinguir usuarios validos probando varios nombres.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`nc -nv $IP 25`

Objetivo y contexto: Conecta directamente por SMTP para leer el banner (revela software/version) e interactuar a mano con VRFY/EXPN.

Resultado esperado: Un banner tipo '220 mail.corp.local ESMTP Postfix'. Desde esta sesión, escribe VRFY usuario y observa el código de respuesta.

Anatomía de los payloads

La primera prueba es `nc -nv $IP 25`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un código 250/252 (usuario existe) frente a 550 (no existe) te deja distinguir usuarios válidos probando varios nombres.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Conecta y observa el banner: a veces ya revela el software y version.
- Prueba VRFY con nombres candidatos (usuarios ya vistos en otras fuentes, nombres comunes).
- Si VRFY está desactivado, prueba EXPN sobre listas de distribución conocidas o comunes.
- Los usuarios confirmados alimentan spraying (fuerza-bruta-online) contra otros servicios del objetivo.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor tiene VRFY/EXPN desactivados (respuesta 'not implemented' o similar) y no hay otra vía de enumerar usuarios por SMTP.
- El puerto pertenece a un relay externo (no al servidor objetivo) sin relación con el dominio que atacas.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El servidor de correo a veces confirma si un usuario existe (VRFY/EXPN) o filtra software en el banner. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(smtp-enum\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion de SNMP

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

SNMP (puerto 161/UDP) es un protocolo de gestion y monitorizacion de dispositivos de red. Se autentica con una 'community string' en vez de usuario/password; si el dispositivo sigue usando la que trae por defecto ('public' para lectura), cualquiera que la conozca puede consultar una cantidad enorme de informacion del sistema: procesos en ejecucion (con sus argumentos, que a veces llevan credenciales), interfaces de red, software instalado, y en Windows a veces hasta usuarios locales.

La community string por defecto es publica y conocida (esta en cualquier manual); si el administrador no la cambia, SNMP queda abierto a cualquiera que sepa probarla. No es un fallo del protocolo, es no cambiar una credencial por defecto, igual que smb-enum con sesiones nulas.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando el recon muestra 161/UDP abierto (nmap -sU lo detecta, un escaneo TCP normal NO lo vera). Muy rentable en maquinas de red/infraestructura.

Cómo identificarla

- Puerto 161/UDP abierto (solo visible con un escaneo UDP, no en el TCP por defecto).
- snmpwalk con 'public' devuelve datos en vez de timeout.
- Los datos incluyen procesos con linea de comandos completa, a veces con credenciales en los argumentos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Consulta con snmpwalk usando la community 'public' (la mas comun por defecto).

Evidencia esperada: Una respuesta con datos del sistema (nombre de host, procesos, interfaces) confirma que la community funciona. Timeout repetido sugiere filtrado UDP o community incorrecta (difícil distinguir sin mas pruebas).

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
snmpwalk -v2c -c public $IP
```

Objetivo y contexto: Vuelca toda la información accesible vía SNMPv2 con la community 'public', la que casi siempre viene por defecto si nadie la cambia.

Resultado esperado: Cientos de líneas con datos del sistema. Busca específicamente procesos (hrSWRunParameters) y su línea de comandos completa: ahí caen credenciales a menudo.

Anatomía de los payloads

La primera prueba es `snmpwalk -v2c -c public $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una respuesta con datos del sistema (nombre de host, procesos, interfaces) confirma que la community funciona. Timeout repetido sugiere filtrado UDP o community incorrecta (difícil distinguir sin más pruebas).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma el puerto con un escaneo UDP (`nmap -sU`); un TCP normal no lo detecta.
- Prueba la community 'public' primero (la más común); si falla, prueba otras por defecto (`private`, `community`) o fuerza bruta si el reto lo sugiere.
- Vuelca toda la información disponible (procesos, software, interfaces) con `snmpwalk`.
- Revisa la línea de comandos de cada proceso: es un sitio clásico donde aparecen credenciales pasadas como argumento.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La community string 'public' es rechazada y no tienes otra candidata que probar.

- El puerto aparece 'open|filtered' de forma persistente sin responder a ninguna consulta (UDP filtrado, no un SNMP mal configurado).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

SNMP con la community string 'public' filtra procesos, usuarios y software instalado sin necesitar credenciales reales. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(snmp-enum\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

TCP frente a UDP

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

TCP confirma una conexión orientada a estado; UDP no establece handshake y requiere sondas específicas.

Un escaneo TCP completo puede dejar fuera DNS, SNMP, TFTP o servicios de voz expuestos por UDP.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando el objetivo parece vacío en TCP o aparecen puertos UDP habituales como 53, 161 o 69.

Cómo identificarla

- open|filtered
- 53/udp
- 161/udp
- sin respuesta TCP

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara un barrido TCP con una sonda UDP sobre puertos esperados y registra estados open|filtered.

Evidencia esperada: Tienes una superficie de red completa y sabes que resultados UDP necesitan una segunda comprobación.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.

5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
nmap -sU -Pn -p 53,69,123,161 $IP
```

Objetivo y contexto: Comprueba UDP de alto valor sin barrer todo a ciegas.

Resultado esperado: Puertos open o open|filtered para profundizar.

Capa 2: prueba documentada

```
nmap -sU -sV -Pn -p <PUERTOS_UDP> $IP
```

Objetivo y contexto: Identifica versiones en los UDP que merecen seguimiento.

Resultado esperado: Banner o respuesta del servicio.

Anatomía de los payloads

La primera prueba es `nmap -sU -Pn -p 53,69,123,161 $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tienes una superficie de red completa y sabes que resultados UDP necesitan una segunda comprobación.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Escanea TCP.
- Sondea UDP prioritario.
- Enumera cada servicio confirmado.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No lo priorices si el alcance solo autoriza TCP y la evidencia confirma que no hay UDP.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Interpretar protocolos y no perder servicios UDP durante el reconocimiento. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(tcp-vs-udp\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.